



DOCUMENTAÇÃO DO FRAMEWORK MQTT RÁDIO LoRa

Tópico 7.3 - Modificação de Parâmetros de Rádio LoRa On the Air

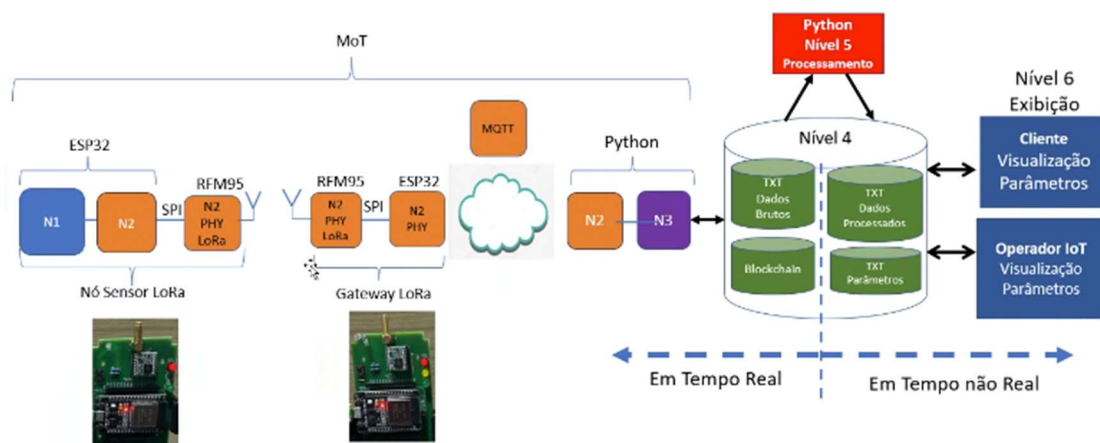
Sumário

1 Considerações Iniciais – Framework MQTT	2
1.1 O que é o protocolo MQTT	3
2 O Framework MQTT	7
2.1 O Gateway LoRa MQTT	9
2.2 O Nível 2-3 com MQTT	10
2.3 Fluxo dos Pacotes de Uplinks - Subida	11
2.4 Fluxo dos Pacotes de Downlink - Descida	14
2.5 Nível 1 – Firmware do Nó Sensor LoRa MQTT	17
2.6 Nível 1 – Firmware do Gateway LoRa MQTT	17
2.7 Python Nível 2 e 3 – Arquivo .N2_N3_MoT_MQTT_Radio.py	19
2.8 Python Nível 4 – Armazenamento em arquivos TXT	22
2.9 Python Nível 5 – Arquivo .N5_Medidas.py	22
2.10 Python Nível 6 – Arquivo .N6_Gerencia_LoRa.py	23

1 Considerações Iniciais – Framework MQTT

O Framework é parte integrante da TpM para a implementação de uma solução IoT. Esse documento trata-se do **Framework MQTT** que tem como diretriz o Framework Básico da TpM, sendo este último o referencial para todos os trabalhos no WissTek-IoT.

Este documento trata-se da apresentação deste Framework MQTT, e as implementações realizadas em relação ao Framework Básico, onde o tráfego de dados entre o Gateway LoRa e o Nível 2_3 (Python), deixa de ser cabado através de uma conexão Serial-USB e passa a ter um Broker MQTT como uma “ponte” entre estes dois elementos de Nível 2, conforme ilustrado abaixo.



Este exemplo de implementação de um Broker MQTT no Framework, permite através da desconexão física (cabo USB), reposicionar o Gateway LoRa, distante da máquina local (onde rodam os Pythons), desde que a sua localidade possua cobertura de rede Wi-Fi com acesso à internet, permitindo inclusive o acesso aos dados dos sensores (Nível 1), bem como o gerenciamento de redes LoRa instaladas remotamente.

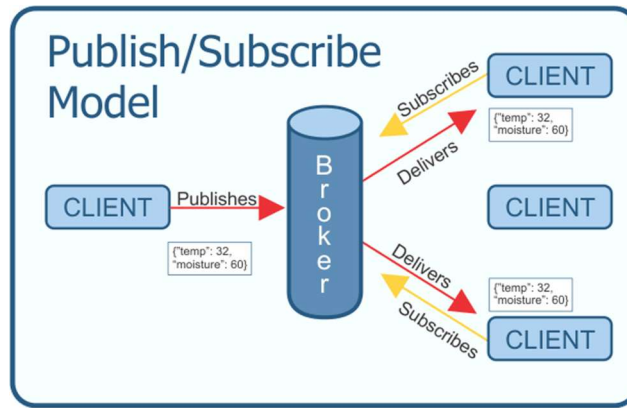
1.1 O que é o protocolo MQTT

O protocolo MQTT (Message Queuing Telemetry Transport) foi inventado em 1999 pela IBM, inicialmente conhecido por IBM MQ Series, a princípio para ser utilizado no setor de petróleo e gás, para atender a necessidade de um protocolo leve, de largura de banda mínima e baixo consumo de bateria para monitorar oleodutos via satélite. Em 2010, a IBM lançou o MQTT 3.1 como um protocolo gratuito e aberto para qualquer pessoa implementar, e em 2013 foi realizada a padronização pelo órgão de especificação Organization for the Advancement of Structured Information Standards (OASIS), que lançou em 2019 uma versão atualizada do MQTT 5, onde foi considerado o **MQTT** como nome oficial do protocolo.

Devido a suas característica de leveza, através de sua implementação com poucas linhas de código e a necessidade de baixo consumo de recursos para as operações de envio e recebimento de mensagens chamadas de Tópicos, o protocolo **MQTT** tem atendido em escalabilidade aplicações em **Soluções em Internet das Coisas (IoT)**, para a troca de informações entre os dispositivos de rede.

[<https://aws.amazon.com/pt/what-is/mqtt/>]

Ele opera no modelo *Publish/Subscribe (Publicação e Assinatura)*, onde dispositivos publicam dados através de Tópicos em servidores intermediários (*brokers*), que os distribuem aos dispositivos de rede inscritos através de assinatura neste Tópico publicado.



O modelo acima de publicação/assinatura ilustra a definição entre clientes e agente (Broker) de uma rede de comunicação MQTT, que requer a implementação uma pilha TCP/IP para se comunicarem através de Tópicos de mensagens. Os clientes nunca se conectam entre si, apenas com o agente broker.

Cliente MQTT - Basicamente, qualquer dispositivo que se comunique por MQTT em uma rede pode ser chamado de dispositivo cliente MQTT. Se o cliente estiver transmitindo Tópicos de mensagens, atuará como publicador, se estiver recebendo mensagens de um Tópico, atuará como assinante ou destinatário.

Agente MQTT - O agente MQTT é o sistema backend chamado de Broker MQTT. Entre as responsabilidades do agente estão:

- Coordenar mensagens dos Tópicos entre os diferentes clientes.
- Receber e filtrar mensagens.
- Identificar os clientes inscritos em cada Tópico de mensagem e enviar as mensagens a eles.
- Autorizar e autenticar clientes MQTT a publicarem e a se inscreverem.
- Transmitir mensagens a outros sistemas para análise posterior.
- Solucionar mensagens e sessões de clientes perdidas.

São especificados 3 tipos de **QoS** (Quality of Service) para o protocolo MQTT:

- **QoS 0**: No máximo uma vez - o cliente publica o Tópico da mensagem sem confirmação de recebimento do Broker, o que não garante que a mensagem foi recebida ou publicada;

```
[Publicador] --- PUBLISH (QoS 0) ---> [Broker]
```

- **QoS 1**: Ao menos uma vez - o cliente publica o Tópico da mensagem com uma simples confirmação ACK de recebimento do Broker, caso não receba ACK, o cliente reenvia a mensagem, o que pode gerar duplicatas de mensagens;

1. [Publicador] --- PUBLISH (QoS 1) ---> [Broker]
2. [Publicador] <--- PUBACK (ACK) ----- [Broker]

- **QoS 2**: Somente uma vez - é o nível mais seguro e complexo. Garante que a mensagem será entregue uma única vez, sem perdas ou duplicações. Utiliza um *handshake* (aperto de mãos) de 4 etapas:

1. [Publicador] --- PUBLISH (QoS 2) -----> [Broker]
2. [Publicador] <--- PUBREC (Received) ----- [Broker]
3. [Publicador] --- PUBREL (Release) -----> [Broker]
4. [Publicador] <--- PUBCOMP (Complete) ----- [Broker]

As bibliotecas PubSubClient (QoS 0) e MQTT (QoS 1) da plataforma Arduino, atendem a implementação do protocolo MQTT sobre a capacidade de processamento do Hardware ESP32 do Kit PKLoRa. É necessário avaliar a capacidade de processamento do Hardware ESP32 para implementação do QoS 2, bem como bibliotecas compatíveis com este nível de implementação.

Todavia o Framework da TpM, através da implementação do protocolo MoT sobre o MQTT, conforme demonstrado em aula, está preparado para uma implementação de QoS através do gerenciamento da rede pelo Framework de MAC centralizada, com o controle sobre as camadas Físicas, MAC, de Rede, de Transporte e de Aplicação dos dispositivos de rede LoRa, desde que implementados os Pacotes de descida (Downlink) e Pacotes de subida (Uplink).

O Framework MQTT implementado possui QoS 1, com confirmação ACK (Acknowledgment) pelo Broker MQTT da publicação dos Tópicos dos Pacotes de Downlink e de Uplink.

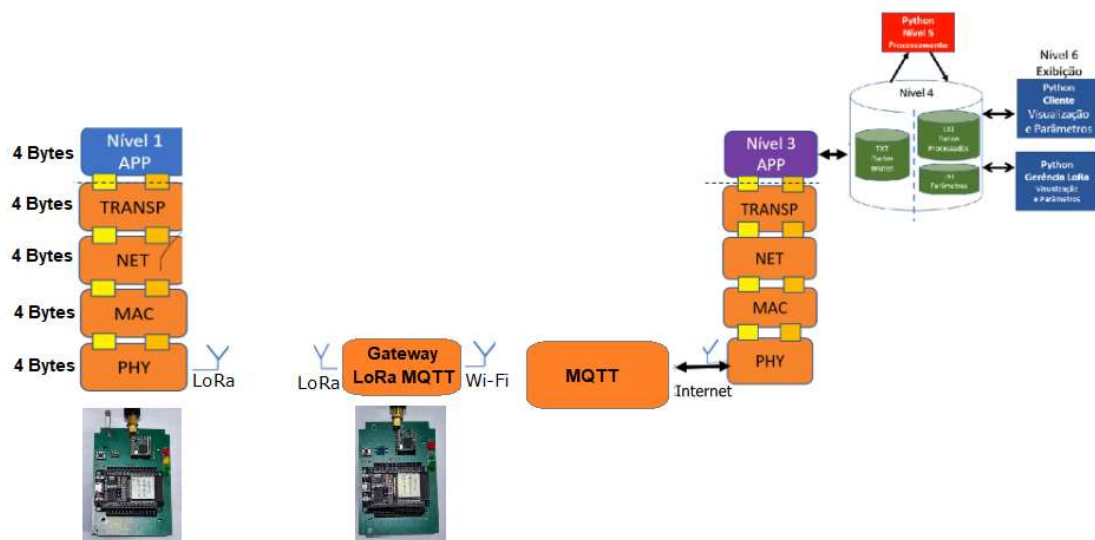
Dois Brokers MQTT foram testados e validados para este Framework, sendo eles Mosquitto e HiveMQ. São Broker MQTT de código aberto (público) e exclusivamente para testes e prototipagem, sem implementação de segurança, Desta forma, dados sensíveis devem ser criptografados, caso sejam publicados em Tópicos nestes Brokers, pois qualquer cliente MQTT que possua acesso as publicações e inscrição nestes Tópicos.

Estes Brokers também fornecem serviços por assinatura, desta forma é possível a adição de chaves de segurança para publicar e se inscrever em cada Tópico criado. Exemplos de Brokers MQTT.



2 O Framework MQTT

O Framework MQTT segue as diretrizes do Framework Básico apresentado como parte integrante da TpM, onde há a adição ao Nível 2 de conectividade um Broker MQTT, com a função de executar uma ponte entre o Gateway LoRa e o elemento de Borda – Níveis 2 e 3.



Assim como o Framework Básico, o Framework MQTT apresenta uma Solução IoT completa, e inclui todos os componentes necessários em cada nível do modelo de referência.

Este Framework MQTT possui a mesma concepção e segue o modelo de código de cores de cada nível do Framework Básico, conforme descritos abaixo:

Nível 1 – faz a medida da luminosidade e comanda o LED amarelo.

Nível 2 – tecnologia de comunicação, que será o LoRa.

Nível 3 – coleta a luminosidade vinda do Nível 1 e envia comando para acionar LED no Nível 1.

Nível 4 – armazenamento de dados em tempo real e em tempo não real.

Nível 5 – processamento dos dados brutos para gerar os dados processados.

Nível 6 – exibição de gráfico da luminosidade e entrada de parâmetro para acionar o LED amarelo.

Este documento, tem por objetivo detalhar exclusivamente a principal diferença entre os Frameworks Básico e MQTT, a adição ao Nível 2 de conectividade, de Broker MQTT, e principalmente o novo fluxo dos dados da rede, através do tráfego dos Pacotes de descida (Downlink) e dos Pacotes de subida (Uplink).

Todos os demais Níveis, permanecem idênticos, assim como também se mantém o tamanho dos Pacotes em 20 bytes, a alocação na pilha de protocolo MoT e funções de cada byte dos Pacotes de DL e UL.

Esta implementação altera exclusivamente a camada física (PHY) do Nível 2 de conectividade entre o Gateway LoRa e a Borda de Níveis 2-3, desconectando-os fisicamente a conexão Serial-USB, e adicionando no lugar o Broker MQTT. As demais camadas e funções da pilha do protocolo MoT, também se mantêm idênticas a do Framework Básico, assim não tem interferência sobre a rede LoRa da Solução IoT proposta no Framework Básico.

2.1 O Gateway LoRa MQTT



O Gateway LoRa MQTT, possui conectividade a redes Wi-Fi (somente redes 2.4GHz) através de seu Hardware, um ESP32 Dev Module. Assim, deve estar posicionado em ambientes com este tipo de cobertura e com acesso à Internet, para que consiga se conectar a um Broker MQTT, para que seja possível o fluxo de dados entre os elementos Gateway LoRa ao Elemento de Borda.

Os seguintes parâmetros devem ser modificados no Firmware do Gateway LoRa, através da plataforma Arduino antes do Upload:

Escolha do KiT LoRa entre Módulo de processamento com ESP32 ou NodeMCU – ESP12E

```
// ESCOLHA DO HARDWARE PKLORA ESP32 OU NODEMCU

// Hardware - Configuração da Seleção do Tipo de Hardware (ESP32 ou
// NODEMCU)
// Remova o comentário da linha referente à placa que você está usando no
// momento e comente a outra

// #define PKLORA_ESP32 // HARDWARE COM ESP32
#define PKLORA_NODEMCU // HARDWARE COM ESP12E - NODEMCU
```

Redes Wi-Fi na // Configuração das redes Wi-Fi 2.4GHz disponíveis

```
void conectar_wifi_multi()
// Cadastre quantas redes você quiser (SSID, Senha) para fallback

wifiMulti.addAP("SSID_WIFI_1", "senha_1");
wifiMulti.addAP("SSID_WIFI_2", "senha_2");
```

Broker MQTT - escolha entre os Brokers Mosquitto ou HiveMQ

```
// Configurações do Broker Mosquitto (Usando o broker público oficial)
const char* MQTT_BROKER = "test.mosquitto.org";

// Configurações do Broker HiveMQ (Usando o broker público oficial)
//const char* MQTT_BROKER = "broker.hivemq.com";
```

Tópicos de Downlink (TOPIC_DL) e Uplink (TOPIC_UL) – Definição do nome

```
const char* TOPIC_DL      = "mot_lora_SEU_RA/gateway/downlink";
const char* TOPIC_UL      = "mot_lora_SEU_RA/gateway/uplink";
```

2.2 O Nível 2-3 com MQTT

No Nível 2-3 com MQTT, também deverá ser feita a escolha do Broker de preferência, deixe-o sem a função # de comentário do Python, e para o outro Broker que não será utilizado comente a linha com a função #:

Utilizar a função de # do Python para comentar o Broker não utilizado.

```
# ===== Configurações MQTT =====
BROKER      = "test.mosquitto.org"
#BROKER      = "broker.hivemq.com"
PORTA_MQTT  = 1883
```

Também, edite a nomenclatura dos Tópicos de Publicação e de Inscrição, renomeando os Tópicos de DL e UL de acordo com o programado no Gateway LoRa, como o seu RA ou outra nomenclatura, como SOBRENOME de forma a evitar que mais um aluno realize Publicações e Inscrições no mesmo Tópico, o que poderá implicar em conflito de medidas, já que ambos estarão utilizando os mesmos Tópicos.

```
# MODIFIQUE O TOPIC_DL E TOPIC_UL de acordo com SEU_NOME
TOPIC_DL      = "mot_lora_SEU_RA/gateway/downlink"
TOPIC_UL      = "mot_lora_SEU_RA/gateway/uplink"
```

OBS.: Estas configurações devem ser as mesmas do Gateway LoRa MQTT

2.3 Fluxo dos Pacotes de Downlink – Descida

Os Parâmetros de Rádio LoRa e a quantidade de medidas são inseridos no Nível 6. Um comando de INICIAR é requisitado:

FEE230 - NÍVEL 6 - GERÊNCIA LORA

LoRa Site Survey - Parâmetros de Rádio

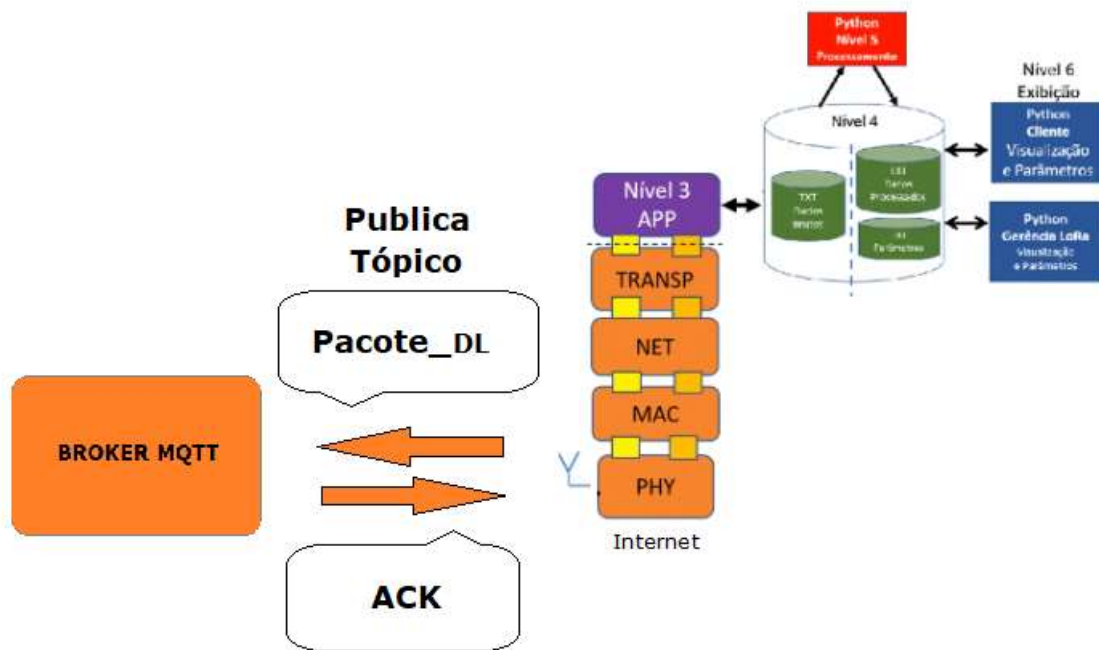
Configurações LoRa

Qtde. de Medidas	100
Tempo de Rádio Em segundos	8
Spreading Factor 7 a 12	12
Bandwidth 125, 250, 500 kHz	125
CodingRate 5 a 8 => 4/5, 4/6, 4/7, 4/8	8
Potência de Rádio 2 a 20dBm	20

INICIAR PARAR

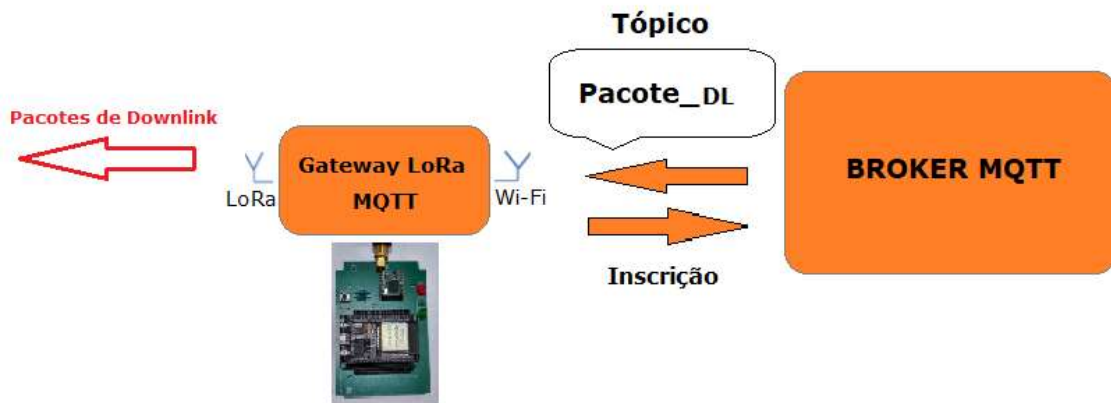
LoRa Site Survey - TESTE PARADO
LoRa Site Survey - ESTADO PARADO

O Nível 6 captura os valores dos parâmetros de Rádio LoRa: Spreading Factor, Bandwidth, Coding Rate e Potência de Transmissão, e os armazena no arquivo PARAMETROS.txt no Nível 4.



No Nível 2-3 (Python), os Pacotes de Downlink são parametrizados (leitura do arquivo PARAMETROS.txt do Nível 4) e montados para serem transmitidos pelo Framework em direção ao Nó Sensor, executando as seguintes funções:

- Conecta-se a um Broker MQTT via rede Wi-Fi ou Ethernet com acesso à Internet;
- Lê parâmetros do Nível 4 (comando do LED Amarelo ativado/desativado), tempo entre pacotes, configuração de rádio LoRa, quantidade de medidas, contador de Pacotes DL, e monta o Pacote de Downlink seguindo o protocolo MoT no payload do Tópico de Downlink TOPIC_DL e o publica no Broker;
- Recebe ACK (confirmação da publicação) do broker MQTT (QoS 1);



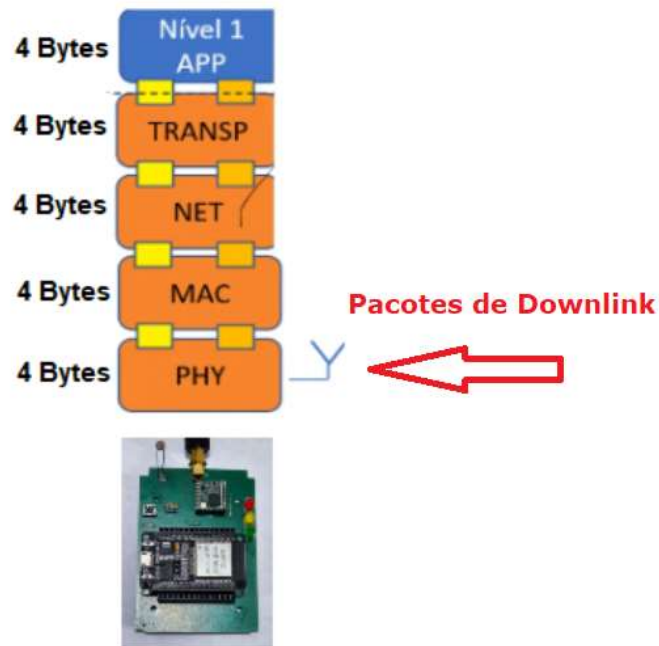
O Tópico de Uplink TOPIC_DL publicado pelo Nível 2-3 no Broker MQTT contendo em seu payload o conteúdo o próprio Pacote_DL, é então disponibilizado pelo Broker MQTT aos clientes MQTT inscritos neste Tópico.

O Gateway LoRa MQTT, deverá estar conectado ao Broker como seu cliente MQTT, e inscrito neste Tópico Pacote_DL, para que tenha a permissão de realizar a sua leitura.

Após lido o Tópico Pacote_DL, o Gateway LoRa MQTT extrai do payload, decodifica e armazena o Pacote DL.

Pacote_DL contém os valores de parâmetros de Rádio LoRa: Spreading Factor, Bandwidth, Coding Rate e Potência de Transmissão, bem como a instrução de modificação dos parâmetros de Rádio LoRa.

Em seguida, o Gateway LoRa escreve o Pacote DL pela SPI ao RFM95 (rádio LoRa) para ser transmitido, endereçado ao Nó Sensor LoRa.



O Pacote de Downlink é lido pelo Nó Sensor LoRa, que possui como execução principal no void loop () a função de escutar o Rádio LoRa, de forma a identificar chegada de novos Pacotes DL, conforme abaixo:

- Caso identificado novo Pacote DL, via comunicação SPI com o RFM95, realiza a leitura e armazena o Pacote_DL - Camada PHY;
- Faz a leitura dos valores de parâmetros de Rádio LoRa: Spreading Factor, Bandwidth, Coding Rate e Potência de Transmissão, mede a RSSI e SNR do Pacote DL recebido e os armazena;
- Realizar leitura do tempo entre pacotes e comando de modificação dos parâmetros de Rádio LoRa - Camada MAC;
- Caso Pacote Downlink esteja endereçado a este Nó Sensor - análise na Camada de Rede, chama a Camada de Transporte, caso contrário, descarta o pacote;

Na Camada de Transporte realiza a contagem de Pacotes DL;

Na Camada de Aplicação atua sobre o LED Amarelo - lê byte 16;

Protocolo Mot => Pacote_DL - Pacote de Downlink - Descida de Pilha

Estado de Máquina LSS [MAC Centralizada - Nível 2-3] :

```
PacoteDL[7] = 3; // Realiza um Teste de Enlace

PacoteDL[7] = 1; // Comanda Modificação de Parâmetros de Rádio

PacoteDL[7] = 4; // Indica Execução das Medidas de testes de LSS

PacoteDL[7] = 5; // Indica Fim das Medidas de testes de LSS

PacoteDL[7] = 10; // Indica Perda de Enlace para o Gateway


// Variáveis de configurações de rádio LoRa e comando MAC


// Bytes da Camada Física

valor_novo_spreadingfactor = PacoteDL[0]; // LoRa Spreading Spectrum

valor_novo_bandwidth = PacoteDL[1]; // LoRa Bandwidth

valor_novo_codingrate = PacoteDL[2]; // LoRa CodingRate

valor_novo_potencia_radio = PacoteDL[3]; // LoRa Potência Transmissão


//Bytes da camada MAC

tempo_radio = PacoteDL[6];

recebe_comando_nova_radio = PacoteDL[7];
```

Tabela Verdade Framework LSS com mudança de parâmetros de Rádio Lora:

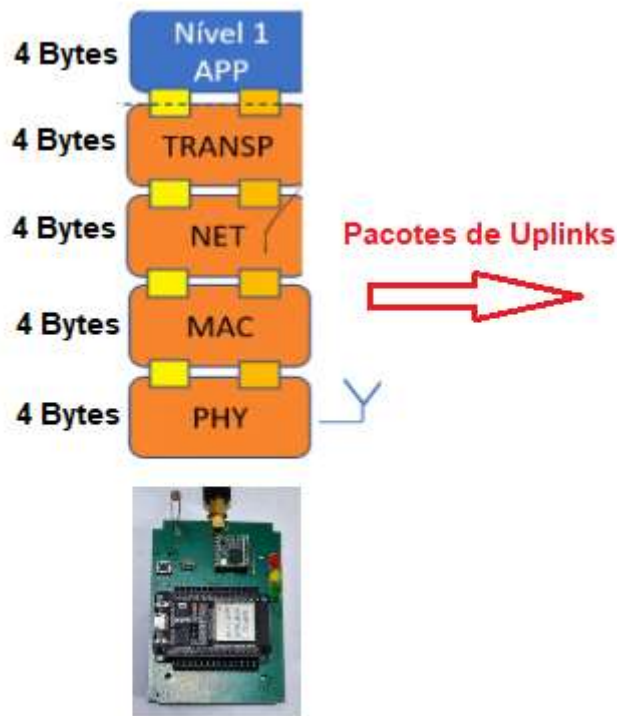
Pacote Downlink - Pacote de Descida da Pilha => MAC Centralizada (N2_N3) => Gateway LoRa
=> Nó Sensor

<code>PacoteDL[7]</code>	Gateway [PHY]	Nó Sensor [MAC]	Descrição
3 => Estado de Máquina <code>PacoteDL[7] = 3</code>	Se, <code>PacoteDL[7] = 3</code> Confirma recebimento: <code>confirma_novo_radio_base = 3</code> Envia <code>PacoteDL[7] = 3</code> ao Nó Sensor	Se, <code>PacoteDL[7] = 3</code> Confirma recebimento: <code>confirma_novo_radio_sensor = 3</code>	Teste de Enlace
1 => Estado de Máquina <code>PacoteDL[7] = 1</code>	Se, <code>PacoteDL[7] = 1</code> Confirma recebimento: <code>confirma_novo_radio_base = 2</code> Envia <code>PacoteDL[7] =</code>		Comando de Mudança de Configuração de Rádio LoRa

	1 ao Nó Sensor	Se, PacoteDL[7] = 1 Confirma recebimento: confirma_novo_ra dio_sensor = 2	
4 => Estado de Máquina PacoteDL[7] = 4	Se, PacoteDL[7] = 4 Confirma recebimento: confirma_novo_ra dio_base = 4 Envia PacoteDL[7] = 4 ao Nó Sensor	Se, PacoteDL[7] = 4 Confirma recebimento: confirma_novo_ra dio_sensor = 4	Execução do Site Survey Controla Números de Pacotes_DL perdidos
5=> Estado de Máquina PacoteDL[7] = 5	Se, PacoteDL[7] = 5 Confirma recebimento: confirma_novo_ra dio_base = 5 Envia PacoteDL[7] = 5 ao Nó Sensor	Se, PacoteDL[7] = 5 Confirma recebimento:	Indicação do Último Pacote de Medidas do Site Survey

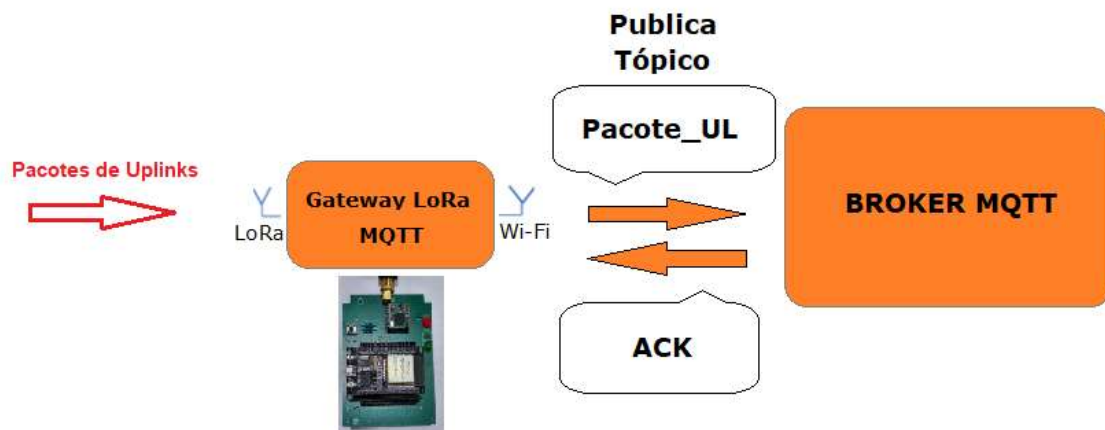
		<code>confirma_novo_ra dio_sensor = 5</code>	
10 => Estado de Máquina <code>PacoteDL[7] = 10</code>	Se, <code>PacoteDL[7] = 10</code> Confirma recebimento: <code>confirma_novo_ra dio_base = 10</code> Não envia <code>PacoteDL</code>	Não recebe <code>PacoteDL</code> Não tem processamento	Modifica Parâmetros de Rádio do Gateway para LoRa Long Range caso Perda de Enlace (Nó Sensor reiniciou por exemplo)

2.4 Fluxo dos Pacotes de Uplinks - Subida



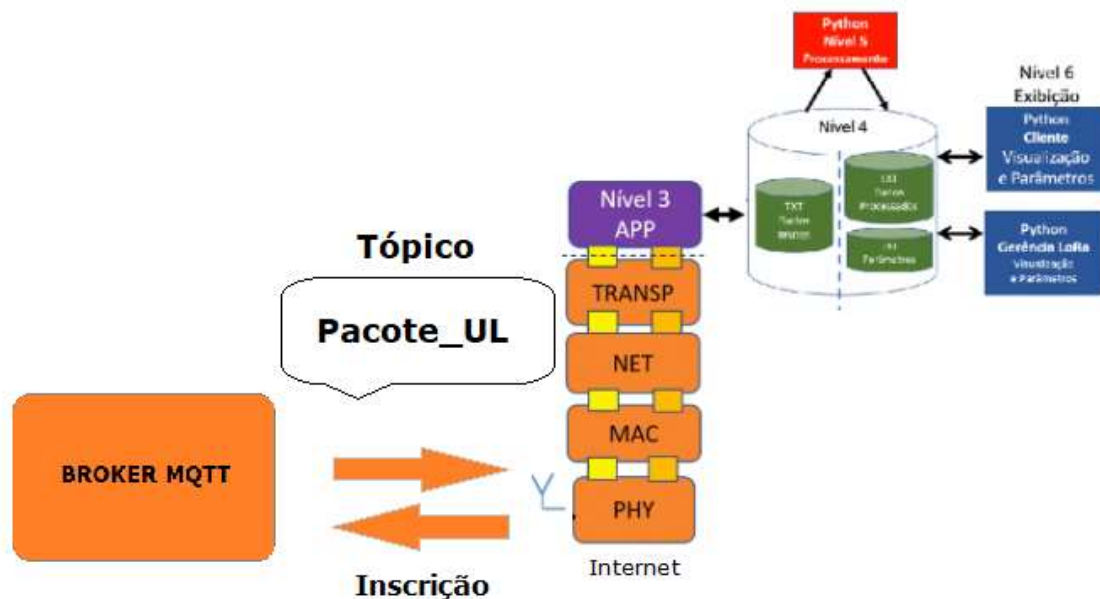
Assim como no Framework MQTT, os Pacotes de Uplink (Subida) são montados via protocolo MoT pelo Nó Sensor e são transmitidos ao Gateway via rede LoRa, com os seguintes dados:

- Valor da medida de luminosidade (LDR) – Camada de Aplicação;
- Contadores de Downlink e Uplink – Camada de Transporte;
- Endereçamento do Nó Sensor e Gateway – Camada de Rede;
- Feedback da Configuração dos Parâmetros de Rádio - Camada MAC;
- Valores medidos de RSSI e SNR do Pacote DL – Camada Física;



Os Pacotes de Uplink, então transmitidos pelo Nó Sensor LoRa, são recebidos e processados pelo Gateway LoRa MQTT, que executa as seguintes funções:

- Faz a recepção via rede Lora e armazena o novo Pacote de Uplink;
- Controla e incrementa o Feedback da Configuração dos Parâmetros de Rádio do Nó Sensor;
- Adiciona os valores medidos de RSSI e SNR ao Pacote de Uplink;
- Monta o Pacote de Uplink via protocolo MoT como Payload do Tópico de Uplink TOPIC_UL e o publica no Broker;
- Recebe ACK (confirmação da publicação) do broker MQTT (QoS 1);



O Tópico de Uplink TOPIC_UL publicado pelo Gateway LoRa MQTT contendo em seu payload o conteúdo o próprio Pacote_UL, é então disponibilizado pelo Broker MQTT neste Tópico aos clientes MQTT inscritos neste Tópico.

O Nível 2-3 (Python) do Framework, deverá estar conectado ao Broker como seu cliente MQTT, e inscrito neste Tópico Pacote_UL, para que tenha a permissão de realizar a sua leitura.

Após lido o Tópico Pacote_UL, o Nível 2-3 extrai do payload, e o decodifica e armazena o Pacote UL no Nível 4 do framework (dados brutos).

À partir deste processo de armazenamento, o Framework segue processando os dados brutos pelo Nível 5, e o Nível 6 realiza a exibição dos dados processados pelo Nivel 5, armazenados também no Nivel 4, exatamente como no Framework Básico.

**Pacote Uplink - Pacote de Subida da Pilha => Nó Sensor => Gateway LoRa
=> MAC Centralizada (N2_N3)**

Nó Sensor [MAC]	Gateway [PHY]	PacoteUL[7]	Descrição
Se, PacoteDL[7] = 3 Confirma recebimento: <pre>confirma_novo_radio_sensor = 3</pre> Envia PacoteUL[7] = 3 ao Gateway	Se, PacoteUL[7] = 3 Confirma recebimento: <pre>confirma_novo_radio_sensor = 3</pre> Se, <pre>confirma_novo_radio_base = 3</pre> E, (&) <pre>confirma_novo_radio_sensor = 3</pre> Envia PacoteUL[7] = 4 a MAC N2_N3	MAC N2_N3: Se, PacoteUL[7] = 4 Confirma recebimento: Comanda aplicar de configurações de Rádio LoRa <pre>comando_novo_radio = 1</pre>	Teste de Enlace

Se, PacoteDL[7] = 1 Confirma recebimento: <pre>confirma_novo_ra dio_sensor = 2</pre> Envia PacoteUL[7] = 2 ao Gateway	Se, PacoteUL[7] = 2 Confirma recebimento: <pre>confirma_novo_ra dio_sensor = 2</pre> Se, <pre>confirma_novo_ra dio_base = 2</pre> E, (&) <pre>confirma_novo_ra dio_sensor = 2</pre> Envia PacoteUL[7] = 3 a MAC N2_N3	MAC N2_N3: Se, PacoteUL[7] = 3 Confirma recebimento: Comanda Início dos Testes de Medidas de Site Survey <pre>comando_novo_rad io = 4</pre>	Comando de Mudança de Configuração de Rádio LoRa
Se, PacoteDL[7] = 4 Confirma recebimento: <pre>confirma_novo_ra dio_sensor = 4</pre> Envia PacoteUL[7] = 4 ao Gateway	Se, PacoteUL[7] = 4	MAC N2_N3: Se, PacoteUL[7] = 6	Execução do Site Survey Controla Números de Pacotes_UL perdidos por timeout

	Confirma recebimento: <pre>confirma_novo_radio_sensor = 4</pre> Se, <pre>confirma_novo_radio_base = 4</pre> E, (&) <pre>confirma_novo_radio_sensor = 4</pre> Envia PacoteUL[7] = 6 a MAC N2_N3	Confirma recebimento: Enquanto não for último Pacote de medidas do Site Survey <pre>comando_novo_radio = 4</pre>	
Se, PacoteDL[7] = 5 Confirma recebimento: <pre>confirma_novo_radio_sensor = 5</pre> Envia PacoteUL[7] = 5 ao Gateway	Se, <pre>confirma_novo_radio_base = 5</pre> Envia PacoteUL[7] = 5 a MAC N2_N3	MAC N2_N3: Se, PacoteUL[7] = 5 Confirma recebimento: Último confirmado Pacote LSS Pausado	Indicação do Último Pacote de Medidas do Site Survey

Não recebe PacoteDL Não tem processamento	Se, PacoteDL[7] = 10 Confirma recebimento: confirma_novo_radio_base = 10 Envia PacoteUL[7] = 10 a MAC N2_N3	MAC N2_N3: Se, PacoteUL[7] = 10 Confirma recebimento: Comanda Novo Teste de Enlace comando_novo_radio = 3	Modifica Parâmetros de Rádio do Gateway para LoRa Long Range caso Perda de Enlace (Nó Sensor reiniciou) Realiza novo Teste de Enlace
--	--	---	---

2.5 Nível 1 – Firmware do Nó Sensor LoRa MQTT

Este Firmware deve ser carregado no Dispositivo do Kit PKLoRa Nó Sensor LoRa, mas antes conforme o Gateway LoRa deve ser escolhido o tipo do módulo de processamento entre ESP 32 ou ESP12E (NodeMCU).

```
// ESCOLHA DO HARDWARE PKLORA ESP32 OU NODEMCU

// Hardware - Configuração da Seleção do Tipo de Hardware (ESP32 ou
// NodeMCU)
// Remova o comentário da linha referente à placa que você está usando no
// momento e comente a outra

// #define PKLORA_ESP32 // HARDWARE COM ESP32
#define PKLORA_NODEMCU // HARDWARE COM ESP12E - NODEMCU
```

Pasta "0_Sensor_LSS_Modules_Vs3"

Nome ^

-  0_Sensor_LSS_Modules_Vs3.ino
-  1_PHY.ino
-  2_MAC.ino
-  3_NET.ino
-  4_TRANSP.ino
-  5_APP.ino
-  Bibliotecas.h

2.6 Nível 1 – Firmware do Gateway LoRa MQTT

Este Firmware deve ser carregado no Dispositivo Gateway LoRa do Kit PKLoRa, mas deve ser previamente configurados os Tópicos de Publicação dos Pacotes de DL e UL, e a conectividade do ESP32 com a Rede Wi-Fi local, conforme descrito abaixo.

Pasta "1_Gateway_LoRa_Config_Radio_MQTT_Modules_QoS1_Vs5"

-  1_Gateway_LoRa_Config_Radio_MQTT_Modules_QoS1_Vs5.ino
-  1_PHY_MoT.ino
-  Bibliotecas.h

O Broker MQTT público utilizado para as publicações é o MOSQUITTO ou o HIVEQM, não utilize ele para dados sensíveis:

BROKER = "broker.hivemq.com"

PORTA_MQTT = 1883

Modificar os Tópicos de publicações no Broker MQTT de DL e UL, alterando-os com o seu nome ou sobrenome (SEU_RA), de forma a diferenciar as publicações de mesmo Framework de cada aluno no Broker MQTT.

TOPIC_DL = "mot_lora_SEU_RA/gateway/downlink"

TOPIC_UL = "mot_lora_SEU_RA/gateway/uplink"

```

1_Gateway_LoRa_Basic_MQTT.ino  1_PHY_MQTT.ino
35 // =====
36 // 3 - Configurações MQTT
37 // =====
38 const char* MQTT_BROKER = "broker.hivemq.com";
39 const int   MQTT_PORT   = 1883;
40
41 // ***** MODIFIQUE O TOPIC_DL E TOPIC_UL de acordo com NOME do usuário *****
42 const char* TOPIC_DL    = "mot_lora_SEU_NOME/gateway/downlink"; // Python → ESP32
43 const char* TOPIC_UL    = "mot_lora_SEU_NOME/gateway/uplink";   // ESP32 → Python
44 String CLIENT_ID ;      // ID único no broker
45

```

Configuração da sua rede Wi-Fi para conexão do ESP32 à Internet:

- Dentro da função SETUP, edite os campos SSID e Senha, inserindo as redes Wi-Fi (somente redes 2.4GHz), para conectividade do gateway LoRa à Internet e ao broker MQTT , através de sua rede Wi-Fi. Insira a quantidade de redes necessárias para casos de fallback.

```

1_Gateway_LoRa_MQTT_V0_0.ino  1_PHY_MQTT.ino
112 // Inicia Wi-Fi
113 // Cadastre quantas redes Wi-Fi necessárias (para Fallback) (SSID, Senha)
114 wifiMulti.addAP("SSID_1", "Senha_2");
115 wifiMulti.addAP("SSID_2", "Senha_2");
116 wifiMulti.addAP("SSID_3", "Senha_3");
117 wifiMulti.addAP("SSID_4", "Senha_4");
118

```

Somente após as modificações dos Tópicos de DL e UL e configuração da rede Wi-Fi com Internet, este Firmware deverá ser carregado no Dispositivo Gateway LoRa do Kit PKLoRa.

2.7 Python Nível 2 e 3 – Arquivo .N2_N3_MoT_MQTT.py

Nesta aplicação, de Níveis 2-3 , é realizado os testes LoRa.

Aqui, cada aluno deve modificar a nomenclatura dos Tópicos DL e UL a serem publicados e lidos no Broker MQTT.

O Broker MQTT público utilizado para as publicações é o MOSQUITTO ou HIVEQM:

```
BROKER      = "broker.hivemq.com"
```

```
PORTA_MQTT  = 1883
```

Modificar/Alterar os Tópicos de publicações no Broker MQTT de DL e UL, alterando (SEU_NOME) com o seu nome ou sobrenome, de forma a diferenciar as publicações de mesmo Framework de cada aluno no Broker MQTT.

```
TOPIC_DL    = "mot_lora_SEU_RA/gateway/downlink"
```

```
TOPIC_UL    = "mot_lora_SEU_RA/gateway/uplink"
```

obs: Os tópicos devem ter os mesmos nomes que estão configurados no Gateway.

```
# ===== Configurações MQTT =====
BROKER      = "broker.hivemq.com"
PORTA_MQTT  = 1883

# MODIFIQUE O TOPIC_DL E TOPIC_UL de acordo com SEU NOME
TOPIC_DL     = "mot_lora_SEU_NOME/gateway/downlink"  # N2_N3 publica - Gateway PKLoRa assina
TOPIC_UL     = "mot_lora_SEU_NOME/gateway/uplink"    # Gateway PKLoRa assina - N2_N3 assina
```

Caso alterada as configurações de rádio LoRa nos Firmwares dos Dispositivos de rede LoRa - Nó Sensor e Gateway, alterar o valor da variável Tempo_entre_pacotes, conforme o tempo de rádio LoRa no ar da rede.

```
# ===== Parâmetros de rede / camadas =====
ID_sensor    = 1
ID_gateway   = 0
Tempo_entre_pacotes = 2  # segundos (colocado no byte 4 do DL)
```

Ao executar o Python (Run => F5), a aplicação irá se conectar ao Broker público MQTT HIVEQM através da porta 1883, e se inscrever nos tópicos.

Faz a leitura constante do arquivo PARAMETROS.txt do Nível 4 para ler a primeira linha com o INÍCIO das Medidas, o número de medidas e os Parâmetros de Rádio LoRa.

Após identificado o início das medidas e o número de medidas de testes LoRa, a aplicação N2_N3 realiza a publicação dos Pacotes de Downlink no Broker MQTT, e aguarda em seguida, uma publicação do Gateway Lora no Broker MQTT do Pacote de Uplink, gerado pelo Nó Sensor LoRa.

```
[PKLoRa MQTT] N2 N3 Conectando ao broker.hivemq.com via porta 1883
[PKLoRa MQTT] Conectado ao broker HiveMQ com sucesso.
[PKLoRa MQTT] Inscrito no tópico: mot_lora_SEU_NOME/gateway/uplink

===== Gateway PKLoRa - Comunicação MQTT =====

Entre com o número de medidas (0 para sair): 100
Pacote [DL] 001 publicado no broker | LED=0
Pacote [UL] 001 | RSSI_DL=-69.0 dBm | RSSI_UL=-70.0 dBm | SNR_DL=5.2 dB | SNR_UL=5.5 dB | Lum=0 | LED=0
Pacote [DL] 002 publicado no broker | LED=0
Pacote [UL] 002 | RSSI_DL=-70.0 dBm | RSSI_UL=-70.0 dBm | SNR_DL=6.5 dB | SNR_UL=5.8 dB | Lum=0 | LED=0
Pacote [DL] 003 publicado no broker | LED=0
Pacote [UL] 003 | RSSI_DL=-70.0 dBm | RSSI_UL=-70.0 dBm | SNR_DL=5.2 dB | SNR_UL=5.8 dB | Lum=0 | LED=0
Pacote [DL] 004 publicado no broker | LED=0
Pacote [UL] 004 | RSSI_DL=-70.0 dBm | RSSI_UL=-70.0 dBm | SNR_DL=5.8 dB | SNR_UL=6.0 dB | Lum=0 | LED=0
Pacote [DL] 005 publicado no broker | LED=0
Pacote [UL] 005 | RSSI_DL=-70.0 dBm | RSSI_UL=-70.0 dBm | SNR_DL=5.8 dB | SNR_UL=5.8 dB | Lum=0 | LED=0
Pacote [DL] 006 publicado no broker | LED=0
Pacote [UL] 006 | RSSI_DL=-69.0 dBm | RSSI_UL=-69.0 dBm | SNR_DL=5.5 dB | SNR_UL=5.5 dB | Lum=0 | LED=0
Pacote [DL] 007 publicado no broker | LED=0
Pacote [UL] 007 | RSSI_DL=-69.0 dBm | RSSI_UL=-70.0 dBm | SNR_DL=5.5 dB | SNR_UL=5.8 dB | Lum=0 | LED=0
Pacote [DL] 008 publicado no broker | LED=0
Pacote [UL] 008 | RSSI_DL=-69.0 dBm | RSSI_UL=-69.0 dBm | SNR_DL=5.5 dB | SNR_UL=6.0 dB | Lum=0 | LED=0
Pacote [DL] 009 publicado no broker | LED=0
Pacote [UL] 009 | RSSI_DL=-70.0 dBm | RSSI_UL=-70.0 dBm | SNR_DL=5.5 dB | SNR_UL=5.8 dB | Lum=0 | LED=0
Pacote [DL] 010 publicado no broker | LED=0
Pacote [UL] 010 | RSSI_DL=-69.0 dBm | RSSI_UL=-70.0 dBm | SNR_DL=5.5 dB | SNR_UL=6.2 dB | Lum=0 | LED=0
Pacote [DL] 011 publicado no broker | LED=0
Pacote [UL] 011 | RSSI_DL=-69.0 dBm | RSSI_UL=-70.0 dBm | SNR_DL=5.2 dB | SNR_UL=5.8 dB | Lum=0 | LED=0
Pacote [DL] 012 publicado no broker | LED=0
Pacote [UL] 012 | RSSI_DL=-70.0 dBm | RSSI_UL=-70.0 dBm | SNR_DL=5.2 dB | SNR_UL=5.8 dB | Lum=0 | LED=0
```

Um “timeout=Tempo_entre_pacotes” monitora o intervalo de tempo entre as publicações dos pacotes de Downlink e Uplink, e caso uma nova publicação não ocorra neste intervalo de tempo, é contabilizado como um Pacote Perdido – Pacote UL não recebido.

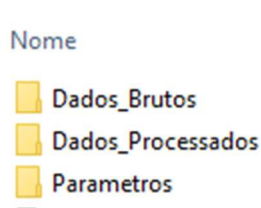
```
*IDLE Shell 3.11.9*
File Edit Shell Debug Options Window Help
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Temp\Pos-Doc\Frameworks\Artigo\Framework_DL_UL_MQTT_PAR\LoRa_Survey_MQTT_Radio\2_N2_N3_Python\N2_N3_LSS_MQTT_Vs6_ToA.py
[MQTT] Conectando ao broker MQTT - porta 1883...
[MQTT] Conectado ao Broker MQTT com sucesso.
[MQTT] Inscrito no tópico: mot_lora_194104/gateway/uplink (QoS1)

===== Gateway LoRa - Comunicação MQTT =====
LSS pausado
LSS pausado
LSS pausado
LSS pausado
LSS pausado
LSS pausado
LSS pausado
```

2.8 Python Nível 4 – Armazenamento em arquivos TXT

Arquivos de LOG, igualmente ao Framework MQTT são gerados conforme a execução de cada teste, para armazenamento das medidas dos dados brutos do Nó Sensor (Pacotes DL e UL em formato TXT), de dados Processados, para processamento e leitura nos Níveis superiores, em formato TMP, e de Parametros, em formato TXT.

Estrutura de arquivos do Nível 4:



2.9 Python Nível 5 – Arquivo N5_Gerencia_SNR_Vs6.py

Nesta aplicação, de Nível 5 – Processamento / Abstração, são lidos os arquivos de Dados Brutos (Pacotes de DL e UL) e calculados os valores de RSSI e SNR dos Pacotes de Downlink e Uplink, da Taxa de Sucesso de Pacotes PSR, e da taxa de transmissão de dados em [bps] teórica, de acordo com os parâmetros de rádio LoRa e real, proporcional a PSR.

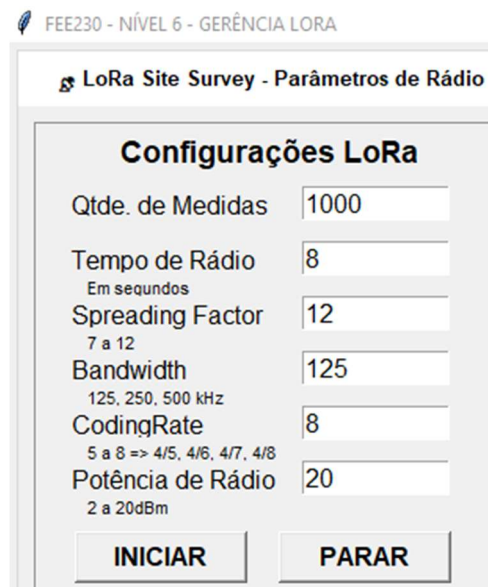
```

Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>> = RESTART: C:\Temp\Pos-Doc\Frameworks\Artigo\Framework_DL_UL_MQTT_PAR\LoRa_Survey\MQTT_Radio\4_N5_Processamento\N5_Gerencia_SNR_Vs6.py
Arquivo = ..\3_N4_Armazenamento\Dados_Brutos\Rodada_Teste_2024_05_29_11-16-39.txt | Pacotes = 100 | Recebidos = 100 | PSR = 100.0 | RSSI Downlink = -20.0 |
RSSI Uplink = -17.5 | SNR Downlink = 5.0 | SNR Uplink = 5.0 | RSSI DL min/max = -22.0 -19.0 | RSSI UL min/max = -20.5 -17.5 | SNR DL min/max = 4.5 5.25 | SNR
UL min/max = 4.5 6.0 | Taxa Canal Teórica (bps) = 21875.0 | Taxa Canal Calculada (bps) = 21875.0 | Pacotes Perdidos = 0
Arquivo = ..\3_N4_Armazenamento\Dados_Brutos\Rodada_Teste_2024_05_29_11-16-39.txt | Pacotes = 100 | Recebidos = 100 | PSR = 100.0 | RSSI Downlink = -20.0 |
RSSI Uplink = -17.5 | SNR Downlink = 5.0 | SNR Uplink = 5.0 | RSSI DL min/max = -22.0 -19.0 | RSSI UL min/max = -20.5 -17.5 | SNR DL min/max = 4.5 5.25 | SNR
UL min/max = 4.5 6.0 | Taxa Canal Teórica (bps) = 21875.0 | Taxa Canal Calculada (bps) = 21875.0 | Pacotes Perdidos = 0
Arquivo = ..\3_N4_Armazenamento\Dados_Brutos\Rodada_Teste_2024_05_29_11-16-39.txt | Pacotes = 100 | Recebidos = 100 | PSR = 100.0 | RSSI Downlink = -20.0 |
RSSI Uplink = -17.5 | SNR Downlink = 5.0 | SNR Uplink = 5.0 | RSSI DL min/max = -22.0 -19.0 | RSSI UL min/max = -20.5 -17.5 | SNR DL min/max = 4.5 5.25 | SNR
UL min/max = 4.5 6.0 | Taxa Canal Teórica (bps) = 21875.0 | Taxa Canal Calculada (bps) = 21875.0 | Pacotes Perdidos = 0
Arquivo = ..\3_N4_Armazenamento\Dados_Brutos\Rodada_Teste_2024_05_29_11-16-39.txt | Pacotes = 100 | Recebidos = 100 | PSR = 100.0 | RSSI Downlink = -20.0 |
RSSI Uplink = -17.5 | SNR Downlink = 5.0 | SNR Uplink = 5.0 | RSSI DL min/max = -22.0 -19.0 | RSSI UL min/max = -20.5 -17.5 | SNR DL min/max = 4.5
  
```

2.10 Python Nível 6 – Arquivo N6_Gerencia_SNR_Radio_Vs9.py

Nesta aplicação, uma tela operacional de Site Survey LoRa é gerada para inserção do número de medias, tempo entre pacotes, Configurações de Rádio LoRa do Site Survey, Spreading Factor, Bandwidth e Coding Rate. A potência de Transmissão do Nó Sensor também é disponibilizada para ser modificada.



FEE230 - NÍVEL 6 - GERÊNCIA LORA

LoRa Site Survey - Parâmetros de Rádio

Configurações LoRa

Qtde. de Medidas

Tempo de Rádio
Em segundos

Spreading Factor
7 a 12

Bandwidth
125, 250, 500 kHz

CodingRate
5 a 8 => 4/5, 4/6, 4/7, 4/8

Potência de Rádio
2 a 20dBm

INICIAR **PARAR**

Uma segunda aba, após o início das medidas, são lidas as amostras de dados processados do Nível 4, com as informações de RSSI e SNR dos Pacotes de Downlink e Uplink, e exibidas em gráficos, e em texto (última leitura) de cada valor.



Uma terceira aba, após o início das medidas, são lidas as amostras de dados processados do Nível 4, com as informações de Taxa de Sucesso de Pacote PSR, e as Taxas de Dados Teórica e Calculada, proporcional a PSR em [bps], do Enlace Rádio LoRa, e exibidas em gráficos, e em texto (última leitura) de cada valor.

